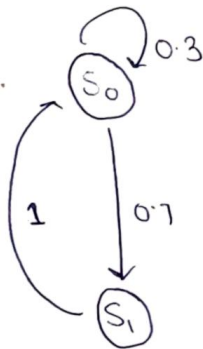


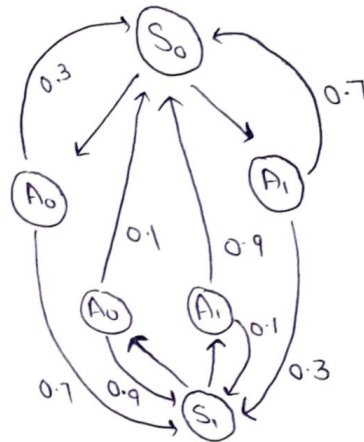
Reinforcement Learning :-

(1)

Markov chain



Markov Decision Process



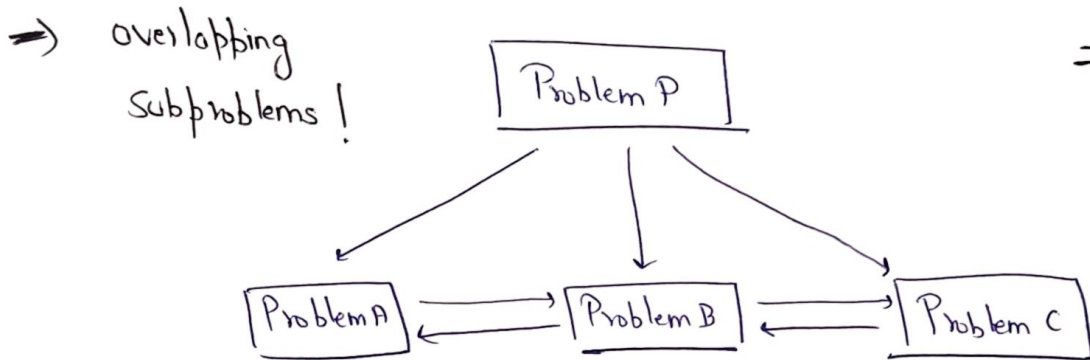
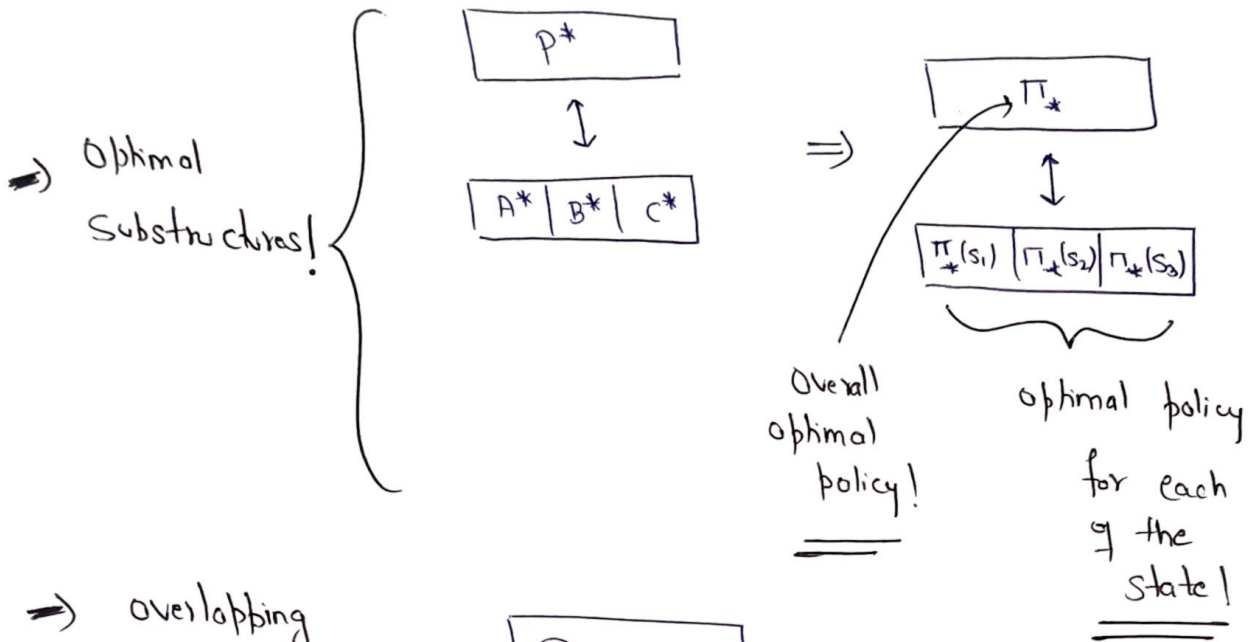
→ Types of MDP's

- 1) Finite and Infinite (# of states and # Actions!) ^{in terms of}
- 2) Episodic and Continuing

→ Dynamic Programming!

∅ Methods that find the soln to a problem by breaking it down into smaller, easier problems!

∅ The optimal solution to all subproblems produces the optimal solution to the original problem!



∴ The solutions to the subproblems are mutually dependent!

⇒ The same as well present in our problem!

$$V_*(s) = \max_a \sum_{s', r} p(s', r | s, a) [r + V_*(s')]$$

$$Q_*(s, a) = \max_a \sum_{s', r} p(s', r | s, a) [r + V \max_{a'} Q_*(s', a')]$$

The solutions to the problems are mutually dependent!

✗ DP turns these equations into update rules —

$$V(s) \leftarrow \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$$

✗ One big limit of the DP is that ... it needs a perfect model of the environment i.e. $p(s', r | s, a)$

✗ DP solves problems using expected values, not trial and error!

→ Value iteration

⇒ we want to find the π_* (optimal policy)

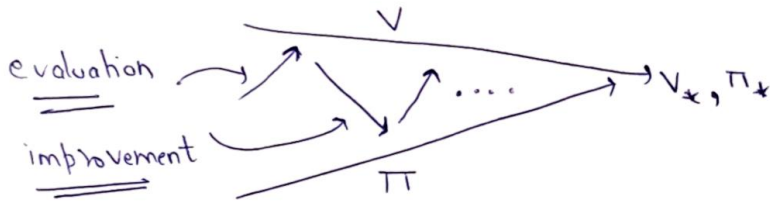
$$\pi_*(s) = \arg \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V_*(s')]$$

.... but for that we have to find V_*

$$\left\{ \begin{array}{l} V(s) \leftarrow \max_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')] \end{array} \right.$$

→ Policy Iteration -

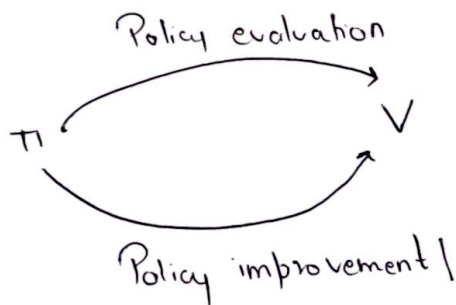
The process that alternatively improves the estimated values and the policy:



$$V(s) \leftarrow \sum_a \pi(s|a) \sum_{s',a} p(s',a|s,a) [r + \gamma V(s')]$$

$$(V_n(s) = \sum_a \pi(s|a) \sum_{s',a} p(s',a|s,a) [r + \gamma V(s')])$$

Generalised Policy Iteration →



Disadvantages of DP -

1) High Computational Cost:

In each sweep we update all the states!

Monte-Carlo Methods

Family of methods that learn optimal $V_*(s)$ or $q_*(s,a)$ values based on samples.

The agent will use a policy π to tackle the task for an entire episode
 $S_0, A_0, R_1, S_1, A_1, R_2, \dots$

They approximate the values by interacting with the environment to generate sample returns and averaging them —

$$V_{\pi}(s) = E_{\pi}[G_t | S_t = s]$$

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}$$

$$V_{\pi}(s) = \frac{1}{N} \sum_{k=1}^N G_{sR}$$

$$q_{\pi}(s,a) = E_{\pi}[G_t | S_t = s, A_t = a]$$

$G_t =$ (same as above)

$$Q_{\pi}(s,a) = \frac{1}{N} \sum_{k=1}^N G_{s,aR}$$

Law of Large Numbers!

- In the limit, this succession of stochastic samples $G_{s_1}, G_{s_2}, \dots, G_{s_n}$ converges to its expected value $V_{\pi}(s)$.

$$P\left(\lim_{n \rightarrow \infty} \bar{G}_s = V_{\pi}(s)\right) = 1$$

→ Advantages of MC :

- ∅ The estimate of a state value does not depend on the rest!
 - ∅ The cost of estimating a state value is independent of the total number of the states!
-

- To improve the policy, we can't use $V(s)$ anymore!

$$T'(s) = \max_a \sum_{s', a} p(s', a | s, a) [r + \gamma V_{\pi}(s')]$$

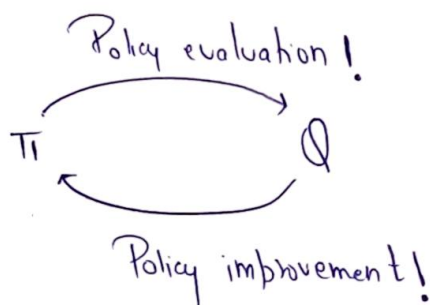
$V(s)$ requires the knowing the effects of taking each action beforehand!

However the environment dynamics are implicit in $q_{\pi}(s, a)$

$$q_{\pi}(s, a) = \sum_{s', a} p(s', a | s, a) [r + \gamma V_{\pi}(s')]$$

→ So we will change the policy as

$$\pi'(s) = \arg \max_a q_{\pi}(s, a)$$



→ $Q(s, a)$ is an estimate!
 (The estimate will improve as we obtain new samples,
 but might not be perfect)

→ How Can we maintain Exploration!

1) Exploring starts }
 $s_0 \sim S, A_0 \sim A$

2) Stochastic Policies } → { on-policy learning }
 $\pi(a|s) > 0, \forall a \in A(s)$ } → { off-policy learning }

⇒ ON-policy Method

ϵ -Greedy policy!

$$\pi(a|s) = \begin{cases} 1 - \epsilon + \epsilon_r & , a = a^* \\ \epsilon_r & , a \neq a^* \end{cases}$$

$$\epsilon_r = \epsilon / |A|$$

with
probability ϵ
Selects random
action!

⇒ Off-policy Method

for finding the optimal actions, -from time to time,
we have to take a suboptimal action
to explore the environment! otherwise we won't
know which actions are the best!

exploratory policy!
 $b(a|s)$

Target policy
 $\pi(a|s)$

Generate the episodes
we are going to update
 $Q(s,a)$ with -

$S_0, A_0, R_1, S_1, A_1, R_2, \dots, R_T$

Policy to be optimized
through $Q(s,a)$ values

$$\pi(s) \leftarrow \arg \max_a Q(s,a)$$

The policy 'b' has to be able to explore all the actions that π it can take! (5)

if $\pi(a|s) > 0$, then $b(a|s) > 0$

The average return will not be approximate the value under ' π ' but under ' b '.

$$E_b[G_t | S_t=s, A_t=a] = q_b(s,a)$$

Importance Sampling! $\div$ statistical technique for estimating the expected value of a distribution by working with samples from another distribution!

$$W_t = \frac{\prod_{k=t}^{T-1} \pi(A_k | S_k)}{\prod_{k=t}^{T-1} b(A_k | S_k)} \left\{ \begin{array}{l} \leftarrow \text{product of all the probabilities of} \\ \text{choosing the actions} \\ \text{taken by the} \\ \text{target policy!} \end{array} \right.$$

choosing the actions taken by the exploratory policy!

By correcting the returns using importance Sampling, we will approximate the value under π .

$$E[w_t G_t | S_t = s] = V_{\pi}(s)$$

Update rule

- 1) ϕ for each $Q(s,a)$, we will keep the list of all observed returns

$$[G_1, G_2, \dots, G_N]$$

- ϕ each time we need to update $Q(s,a)$ we'll recompute the average

$$Q(s,a) \leftarrow \frac{1}{N} \sum_{k=1}^N G_k$$

2)

$$\left. \begin{aligned} Q(s,a) &\leftarrow Q(s,a) + \frac{w_t}{c(s,a)} [G - Q(s,a)] \\ c(s,a) &= \sum_{k=1}^N w_k \end{aligned} \right\}$$

Note
 $\Leftarrow$

Above two Methods are for off-policy.
 for on-policy :-

- 1) Same as above!

2) $Q(s,a) \leftarrow Q(s,a) + \alpha [G - Q(s,a)]$

Constant α
 on-policy MC!

Temporal Difference Methods -

(6)

Combination of MC and DP.

Agent learns from the example!

$$S_0, A_0, R_1, S_1, A_1, \dots, R_T$$

They do not need a model of the environment!

$$T^*(s) = \operatorname{argmax}_a \sum_{s', r} p(s', r | s, a) [r + \gamma V(s')]$$

↓

$$T^*(s) = \operatorname{argmax}_a Q(s, a)$$

They use bootstrapping (i.e. use estimates to produce new estimates)

$$Q(s_t, A_t) \leftarrow \underbrace{Q(s_t, A_t)}_{\text{estimate!}} + \alpha \left(R_{t+1} + \underbrace{\gamma Q(s_{t+1}, A_{t+1})}_{\text{estimate!}} - Q(s_t, A_t) \right)$$

MC methods wait until the return G_t is available before updating $Q(s, a)$:-

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-t} R_T$$

⇒

$$q_{\pi}(s, a) = E_{\pi} \left(r_t \mid S_t = s, A_t = a \right)$$

$$q_{\pi}(s, a) = \sum_{s', s''} p(s', s'' \mid s, a) \left[r_t + V(s') \right]$$

$$= \sum_{s', s''} p(s', s'' \mid s, a) \left[r_t + V \sum_{a'} \pi(a' \mid s') q_{\pi}(s', a') \right]$$

⇒ $q_{\pi}(s_t, A_t)$ can also be estimate as —

$$R_{t+1} + \gamma Q(s_{t+1}, A_{t+1})$$

#

$$\left(R_{t+1} + \gamma Q(s_{t+1}, A_{t+1}) \right) - Q(s_t, A_t)$$

temporal difference!
error.

#

Similar to Constant MC —

$$Q(s_t, A_t) \leftarrow Q(s_t, A_t) + \alpha (r_t - Q(s_t, A_t))$$

estimating —

$$\hat{G}_t = R_{t+1} + \gamma Q(s_{t+1}, A_{t+1})$$

— Allow us to estimate

$Q(s_t, A_t)$ at time 't+1'!

Q-N-Policy Temporal Difference -

(7)

(SARSA)!

ϵ -Greedy policy!

Probability ϵ

$a \sim A(s)$ random

Probability $1-\epsilon$

$a = \underset{a}{\operatorname{argmax}} Q(s,a)$

Update Rule -

$$Q(s_t, A_t) \leftarrow Q(s_t, A_t) + \alpha [R_{t+1} + \gamma Q(s_{t+1}, A_{t+1}) - Q(s_t, A_t)]$$

The ϵ -greedy policy π picks the next action, $\left\{ \begin{array}{l} \text{Used to update } Q(s_t, A_t). \end{array} \right.$

→ Q-learning

Target policy:

$$\pi(s) = \underset{a}{\operatorname{argmax}} Q(s,a)$$

Exploratory policy:

$$b(s)$$

Update Rule -

$$Q(s_t, A_t) \leftarrow Q(s_t, A_t) + \alpha [R_{t+1} + \gamma Q(s_{t+1}, \pi(s_{t+1})) - Q(s_t, A_t)]$$

Updates $Q(s_t, A_t)$ based on the best available action!

$$A_{t+h} = \pi(s_{t+h}) = \underline{\arg \max_a Q(s_{t+h}, a)}$$

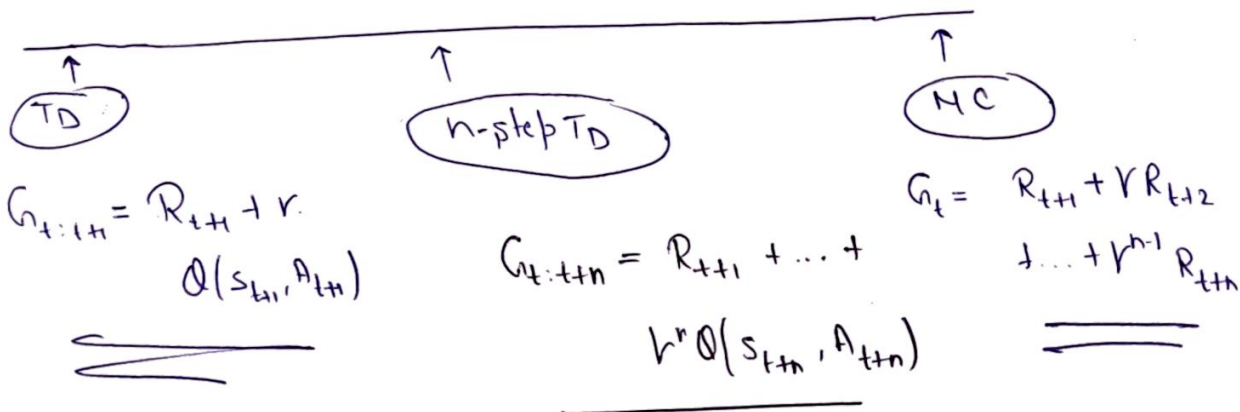
→ N-step bootstrapping —

$$R_{t+1} + \gamma Q(s_{t+1}, A_{t+1}) \quad \left. \vphantom{R_{t+1} + \gamma Q(s_{t+1}, A_{t+1})} \right\} \begin{array}{l} \text{1-step} \\ \text{bootstrapping} \end{array}$$

$$R_{t+1} + \gamma R_{t+2} + \gamma^2 Q(s_{t+2}, A_{t+2}) \quad \left. \vphantom{R_{t+1} + \gamma R_{t+2} + \gamma^2 Q(s_{t+2}, A_{t+2})} \right\} \begin{array}{l} \text{2-step} \\ \text{bootstrapping} \end{array}$$

$$\downarrow$$

$$\underbrace{R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n Q(s_{t+n}, A_{t+n})}_{= G_{t:t+n}} \quad \left. \vphantom{R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n Q(s_{t+n}, A_{t+n})} \right\} \begin{array}{l} \text{n-step} \\ \text{bootstrapping} \end{array}$$



In making the estimate of a value, there are two problems that can negatively affect these estimates i.e. bias and variance!

1) Bias : $Q(S_{t+n}, A_{t+n})$ is an estimate of the future rewards —

$Q(S_{t+n}, A_{t+n})$ introduces bias in an estimate!

Since in the starting $Q(S_{t+n}, A_{t+n})$ is not good estimate i.e. it can be biased

Therefore the higher the 'n', the more heavily discounted estimate $Q(S_{t+n}, A_{t+n})$ by r^n .

i.e. the higher the 'n'
the lower the bias!

2) Variance : # Each reward is a random variable that depends on the state 's' and action 'a' preceding it!

$$\Rightarrow G_t = R_{t+1} + rR_{t+2} + \dots + r^{n-1}R_{t+n}$$

Even if they have the same expected value, G_t samples will be very different from each other!

i.e. the higher the 'n' the higher the variance!

N-step SARSA —

$$Q(s_t, A_t) \leftarrow Q(s_t, A_t) + \alpha [G_{t:t+n} - Q(s_t, A_t)]$$

$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^n Q(s_{t+n}, A_{t+n})$$